ACTIVIDAD PRÁCTICA OBLIGATORIA N°1

MÉTRICAS

AirFlow – Simulador de Gestión Aeroportuaria

Integrantes - Grupo 6:

Rodrigo Nicolás BERTI

rberti@alumno.unlam.edu.ar

Luis Angel CHOQUE

luchoque@alumno.unlam.edu.ar

Mayra Aldana MORANDI

mmorandi@alumno.unlam.edu.ar

Axel Joel RODRIGUEZ FENSKE

arodriguezfenske@alumno.unlam.edu.ar

Gonzalo Ezequiel SANCHEZ

gonzalsanchez@alumno.unlam.edu.ar

Asignatura:

(3658) PROGRAMACIÓN CONCURRENTE

Fecha - Versión:

21/10/2026 – Versión 1.01

Resumen:

El presente trabajo analiza una aplicación concurrente generada mediante inteligencia artificial a partir de un prompt técnico elaborado por el equipo. Como caso de estudio se utilizó una simulación de la operación de una aerolínea, en la cual múltiples hilos coordinan la generación de vuelos, la asignación de aeronaves y la atención de eventos externos. La evaluación combinó rúbricas de funcionalidad y concurrencia con mediciones de CPU y memoria en dos escenarios de carga, además del registro de tokens asociado al proceso de generación y refinamiento. Las rúbricas consignaron 90 puntos sobre 100 en funcionalidad y 75 sobre 100 en concurrencia. La solución incorpora mecanismos de sincronización adecuados, aunque se identificaron aspectos de su implementación que requieren revisión. Esto destaca la necesidad de una revisión humana del código generado por inteligencia artificial, cuyos resultados deben analizarse y validarse antes de considerarlos correctos.

Palabras Clave: *Inteligencia artificial generativa, métricas de software, concurrencia, generación de código*

I. CONTEXTO

El presente trabajo se desarrolla en el marco de la asignatura Programación Concurrente y tiene como área de estudio la Ingeniería de Software, particularmente el desarrollo y evaluación de aplicaciones concurrentes asistidas mediante herramientas de inteligencia artificial generativa. La actividad propone experimentar con herramientas de generación automática de código a partir de prompts técnicos, para posteriormente analizar de manera crítica la solución obtenida mediante métricas relacionadas con funcionalidad, concurrencia, utilización de recursos computacionales y consumo de tokens.

II. INTRODUCCIÓN

La programación concurrente permite estructurar una aplicación de manera que múltiples tareas puedan progresar durante un mismo período de ejecución. Este paradigma puede mejorar la capacidad de respuesta y el aprovechamiento de los recursos computacionales, pero introduce problemáticas específicas vinculadas con la coordinación entre tareas, el acceso a recursos compartidos y la sincronización.

En los últimos años, los modelos de inteligencia artificial generativa comenzaron a utilizarse como herramientas de asistencia para el desarrollo de software. A partir de instrucciones expresadas en lenguaje natural, estos modelos pueden generar implementaciones completas o parciales, proponer soluciones técnicas y realizar modificaciones sucesivas sobre el código generado. Sin embargo, la obtención de código sintácticamente correcto no garantiza necesariamente su correctitud funcional ni, especialmente, su correcto comportamiento en escenarios concurrentes.

El objetivo de este trabajo es analizar la calidad de un programa concurrente generado mediante inteligencia artificial a partir de un prompt técnico elaborado por el equipo. Para ello se evalúan el cumplimiento funcional, la sincronización de tareas, la prevención de deadlocks y condiciones de carrera, y la protección de las secciones críticas. Asimismo, se compara el uso de CPU y memoria en dos escenarios de carga y se registra la cantidad de tokens durante el proceso de generación y refinamiento. El análisis busca identificar fortalezas y limitaciones de la solución obtenida.

III. MÉTODOS

A. Definición del problema

Para realizar el experimento se definió como caso de estudio una simulación concurrente de la operación de una aerolínea. El sistema debe administrar una flota de aviones y asignarlos dinámicamente a diferentes vuelos de acuerdo con su disponibilidad y compatibilidad. Cada avión posee características tales como capacidad de pasajeros,

autonomía, aeropuerto actual y estado operativo. De manera similar, cada vuelo incluye información sobre origen, destino, horario de salida, duración, cantidad de pasajeros, prioridad y estado. Un avión puede ser asignado a un vuelo solamente cuando posee capacidad y autonomía suficientes, se encuentra disponible y está ubicado en el aeropuerto de origen correspondiente. Cuando existen múltiples aeronaves compatibles, el sistema debe seleccionar aquella que permita minimizar el tiempo ocioso o la necesidad de reposicionamiento. El sistema también contempla diferentes niveles de prioridad para los vuelos, entre ellos emergencia, VIP, regular y carga. Con el objetivo de evitar starvation, se especificó un mecanismo de aging mediante el cual la prioridad efectiva de un vuelo aumenta progresivamente en función de su tiempo de espera. Adicionalmente, durante la simulación pueden producirse eventos externos, tales como fallas, mantenimientos, cancelaciones y demoras. Estos eventos pueden modificar el estado de los aviones y vuelos e incluso provocar la necesidad de reasignar un vuelo previamente asignado.

B. Herramienta y modelo utilizado

Herramienta IA: GPT

Modelo: 5.6 Luna

Lenguaje: Phyton

Metodología: Prompt técnico

C. Elaboración del prompt

El equipo elaboró un prompt técnico destinado a establecer tanto los requisitos funcionales de la aplicación como las condiciones relacionadas con su comportamiento concurrente. El prompt fue utilizado como entrada inicial.

Prompt utilizado:

LENGUAJE DE PROGRAMACIÓN:

Python 3.11 o superior, solo biblioteca estándar (threading, queue, logging, dataclasses, enum, random, time).

No usar asyncio ni frameworks externos: la concurrencia debe resolverse con hilos y primitivas explícitas.

No usar números mágicos

métodos o funciones de no más de 15 líneas.

El código debe estar escrito en idioma inglés.

Uso de patrones de diseño.

Espacios en lugar de tabulaciones (2 espacios).

Generar un entorno docker para correr el programa. Detallar los comandos para ejecutarlo en el readme final.

Generar una interfaz html simple en donde pueda ver las operaciones que ocurren

PLATAFORMA:

Aplicación de consola multihilo que simule la operación de una aerolínea.

El tiempo de simulación debe ser acelerado y configurable (por ejemplo, 1 segundo real = 10 minutos simulados).

Los parámetros (cantidad de aviones, vuelos, asignadores, semilla aleatoria, duración) deben poder configurarse por línea de comandos o archivo de configuración.

FUNCIONALIDAD A IMPLEMENTAR:

Desarrollar un sistema que asigne aviones a vuelos de forma dinámica según necesidad y disponibilidad, mientras llegan nuevos vuelos, se completan otros y ocurren eventos imprevistos.

REQUISITOS ESPECÍFICOS:

Entidades:

Avión: id, modelo, capacidad de pasajeros, autonomía, aeropuerto actual, estado.

Vuelo: id, origen, destino, horario de salida, duración, pasajeros, prioridad (emergencia, VIP, regular, carga), estado.

Hilos obligatorios:

Un hilo generador de vuelos (demanda aleatoria).

Al menos 3 hilos asignadores que operen en simultáneo sobre el mismo estado compartido.

Un hilo por avión que simule el vuelo y el turnaround.

Un hilo de eventos externos (fallas, mantenimientos, cancelaciones y demoras aleatorias).

Un hilo monitor/auditor que reporte el estado de la flota y verifique invariantes sin detener el sistema.

Asignación: un avión es compatible con un vuelo si tiene capacidad y autonomía suficientes, está en el aeropuerto de origen y está disponible. Entre varios compatibles, elegir el que minimice tiempo ocioso o reposicionamiento.

Prioridades: los vuelos de mayor prioridad se atienden primero. Para evitar starvation, implementar aging (la prioridad efectiva aumenta con el tiempo de espera).

Eventos imprevistos:

Un avión puede pasar a mantenimiento mientras está asignado o en evaluación. En ese caso, el vuelo vuelve a pendiente y se reasigna.

Un vuelo demorado puede afectar al siguiente vuelo del mismo avión.

Sincronización (obligatorio justificarla en el README):

Lock por avión, y adquisición múltiple siempre en orden global por id para evitar deadlock.

Condition o Semaphore para que los vuelos pendientes esperen la liberación de aviones, sin espera activa.

Prohibido usar time.sleep como mecanismo de sincronización.

Prohibido el patrón check-then-act sin lock (no consultar disponibilidad y asignar en pasos separados).

El monitor debe leer un snapshot consistente del estado.

Máquina de estados:

Avión (mínimo 5 estados): DISPONIBLE, ASIGNADO, EN_VUELO, TURNAROUND, MANTENIMIENTO.

Vuelo (mínimo 5 estados): PENDIENTE, ASIGNADO, EN_CURSO, COMPLETADO, CANCELADO.

Las transiciones válidas deben estar definidas en una tabla y cualquier transición inválida debe lanzar excepción.

Invariantes (verificarlos con aserciones y en el monitor):

Un avión nunca está asignado a dos vuelos con horarios solapados.

Un vuelo tiene como máximo un avión.

Un avión en mantenimiento nunca está asignado ni en vuelo.

La cantidad de aviones por estado suma el total de la flota.

Ningún vuelo compatible espera indefinidamente si hay un avión adecuado disponible.

Logs: usar logging con nivel, timestamp, nombre del hilo e identificadores. Registrar cada cambio de estado (avión AV-03: DISPONIBLE -> ASIGNADO (vuelo FL-120)), cada asignación, reasignación y cancelación, y las violaciones de invariantes como ERROR.

Apagado ordenado: al terminar la simulación o con Ctrl+C, todos los hilos deben finalizar sin quedar bloqueados (usar threading.Event y sentinelas en colas).

Reproducibilidad: toda aleatoriedad debe depender de una semilla configurable.

ENTREGABLES:

Estructura modular de archivos (models.py, scheduler.py, workers.py, monitor.py, main.py).

Un conjunto de tests de estrés (por ejemplo, 50 aviones, 500 vuelos, 8 asignadores) que verifiquen los invariantes al finalizar.

Un README.md que explique las decisiones de sincronización, cómo se previene deadlock y starvation, y cómo ejecutar el programa.

Una breve sección con los riesgos de concurrencia que el diseño evita y cómo se podrían reproducir si se quitara cada mecanismo.

D. Evaluación

La evaluación funcional se realizó utilizando la rúbrica establecida para la actividad. Se consideraron tres dimensiones principales: ejecución del programa sin errores, cumplimiento del objetivo funcional y diseño de la aplicación. La ejecución sin errores permite comprobar que el sistema puede iniciarse y completar su funcionamiento sin fallos que imposibiliten su utilización. El cumplimiento funcional evalúa si la solución implementa efectivamente

los requisitos solicitados en el prompt técnico. Finalmente, el criterio de diseño considera aspectos relacionados con la claridad y facilidad de utilización de la aplicación.

La evaluación de concurrencia se organizó mediante una segunda rúbrica, compuesta por sincronización, deadlocks, condiciones de carrera y exclusión mutua. Estas dimensiones permiten examinar problemas distintos: la coordinación entre tareas, la posibilidad de bloqueos permanentes, la dependencia del resultado respecto del orden de accesos concurrentes y la protección de las secciones críticas.

Para comparar el uso de recursos de CPU y memoria se documentaron dos escenarios. El primero configuró 50 aviones, 30 vuelos y 10 hilos asignadores; el segundo, 500 aviones, 1000 vuelos y 200 hilos asignadores. En ambos se estableció una duración de 60 segundos, un factor de aceleración de 60 minutos simulados por segundo real. Esta comparación permite describir el comportamiento registrado ante dos configuraciones de diferente carga.

Se seleccionó la biblioteca psutil por su integración con Python y porque permite medir el consumo conjunto de CPU y memoria de los hilos que pertenecen al mismo proceso. Además, facilita la automatización de las mediciones mediante un procedimiento común para ambos escenarios de carga. Frente a ps, permite recolectar los datos sin ejecutar comandos externos ni procesar su salida. Por su parte, la combinación ps + jcmd está orientada al análisis de aplicaciones Java, por lo que no resulta adecuada para este programa desarrollado en Python.

El registro de tokens se tomó de la herramienta de generación.

IV. RESULTADOS Y OBJETIVOS

En cuanto a la evaluación funcional, se obtuvo un puntaje de 90 sobre 100, alcanzando la calificación máxima en los criterios de ejecución sin errores y cumplimiento del objetivo funcional. El sistema implementa la simulación de una aerolínea mediante múltiples hilos que gestionan la generación de vuelos, asignación de aeronaves, ejecución de operaciones y eventos externos. Respecto al diseño de la aplicación, se identificaron oportunidades de mejora en la interfaz de usuario, por lo que este criterio recibió una calificación intermedia.

RÚBRICA FUNCIONALIDAD PROGRAMA CONCURRENTE PC (JAVA/PYTHON/KOTLIN)

CRITERIOS	SUFICIENTEMENTE LOGRADO (A)	MEDIANAMENTE LOGRADO (B)	INSUFICIENTEMENTE LOGRADO (C)	Puntaje	Eval
FUNCIONA SIN ERRORES	Compila y funciona sin errores.	Funciona con errores menores o warnings.	Presenta errores que impiden su uso.	A: Hasta 40 pts. B: Hasta 20 pts. C: 0	A
CUMPLIMIENTO DEL OBJETIVO FUNCIONAL	Cumple todos los requisitos funcionales.	Cumple parcialmente los requisitos.	No cumple los requisitos mínimos.	A: Hasta 40 pts. B: Hasta 20 pts. C: 0	A
DISEÑO DE LA APP	Interfaz clara y fácil de usar.	Interfaz funcional con algunos problemas.	Interfaz deficiente o difícil de usar.	A: Hasta 20 pts. B: Hasta 10 pts .C: 0	B
TOTAL				100	90

Por otra parte, la evaluación de concurrencia alcanzó un puntaje de 75 sobre 100. Se destacaron los mecanismos de prevención de deadlocks mediante la adquisición ordenada de locks y la protección de secciones críticas mediante exclusión mutua. Sin embargo, se identificaron aspectos mejorables en la sincronización de tareas y la prevención de condiciones de carrera, los cuales recibieron calificaciones intermedias por presentar riesgos en determinadas situaciones. Aunque se utilizan Condition y Event, la separación entre comprobar la disponibilidad y comenzar la espera puede provocar notificaciones pérdidas y demoras.

RÚBRICA DE CONCURRENCIA

CRITERIOS	SUFICIENTEMENTE LOGRADO (A)	MEDIANAMENTE LOGRADO (B)	INSUFICIENTEMENTE LOGRADO (C)	Puntaje	Eval
SINCRONIZACIÓN	Las tareas están correctamente sincronizadas.	Existen problemas menores de sincronización.	La sincronización es incorrecta o inexistente.	A: hasta 25 pts.B: hasta 15 pts.C: 0 pts.	B = 15
DEADLOCKS	No existen bloqueos y los recursos se liberan correctamente.	Existe riesgo de bloqueo en casos particulares.	Se producen bloqueos durante la ejecución.	A: hasta 25 pts.B: hasta 15 pts.C: 0 pts.	A = 25
CONDICIONES DE CARRERA	Los datos compartidos están correctamente protegidos.	Algunos accesos compartidos presentan riesgos.	Se producen resultados inconsistentes por accesos simultáneos.	A: hasta 25 pts.B: hasta 15 pts.C: 0 pts.	B = 15
EXCLUSIÓN MUTUA	Las secciones críticas están correctamente protegidas.	La protección es parcial o excesiva.	No se protegen correctamente las secciones críticas.	A: hasta 25 pts.B: hasta 15 pts.C: 0 pts.	A = 20
TOTAL				100	75

El consumo de CPU y memoria se comparó en dos simulaciones de 60 segundos. La primera utilizó 50 aviones, 30 vuelos y 10 hilos asignadores; la segunda, 500 aviones, 1000 vuelos y 200 asignadores. El uso de CPU registrado aumentó de 13,03 % a 101,87 %, mientras que la memoria pasó de 25,91 MB a 49,28 MB, un incremento aproximado del 90,2 %.

Codigo	Método medición	Modo CPU	CPUs lógicas	Acción 1	CPU Acc1	Acción 2	CPU Acc2	Memoria Acc1 (MB)	Memoria Acc2 (MB)	MemoriaTotal (MB)
M6	psutil	100%=capacidad total	10	simulación con baja carga: 50 aviones, 30 vuelos, 10 hilos para asignación (60seg de duración)	13,03%	simulación con mayor carga: 500 aviones, 1000 vuelos, 200 hilos para asignación (60seg de duración)	101,87%	25,91	49,28	16384

Simulación con baja carga:

```
docker run --rm \
-p 8080:8080 \
-v "$(pwd)/metrics:/app/metrics" \
airline-simulator \
--duration 60 \
--planes 50 \
```

```
--flights 30 \
--assigners 10 \
--speed 60 \
--seed 7 \
--metrics-file /app/metrics/metrics-complete-a.json
```

Simulación con mayor carga:

```
docker run --rm \
-p 8080:8080 \
-v "$(pwd)/metrics:/app/metrics" \
airline-simulator \
--duration 60 \
--planes 500 \
--flights 1000 \
--assigners 200 \
--speed 60 \
--seed 7 \
--metrics-file /app/metrics/metrics-complete-b.json
```

Durante la generación del código registramos 160.787 tokens y realizamos tres refinamientos para ajustar la solución a los requisitos del prompt. El desarrollo requirió un proceso iterativo de revisión y modificación de la respuesta inicial. La cantidad de tokens permite dimensionar la interacción con la herramienta, pero no representa por sí sola la calidad del código ni garantiza su correcta ejecución.

Código	Método utilizado	Refinamientos	Tokens registrados
M6	Métricas de GTP	3	160787

V. CONCLUSIONES

El desarrollo de este trabajo permitió evaluar la capacidad de la inteligencia artificial generativa para producir una aplicación concurrente a partir de un prompt técnico con requisitos funcionales y restricciones específicas. Los resultados obtenidos demuestran que la herramienta utilizada fue capaz de generar una solución que cumple con los objetivos funcionales planteados, incorporando hilos, mecanismos de sincronización y administración de recursos

compartidos. No obstante, el análisis de concurrencia permitió identificar limitaciones y posibles riesgos que no resultan evidentes únicamente al verificar el funcionamiento general del programa. Esto pone de manifiesto que la correcta ejecución de una aplicación no garantiza la ausencia de problemas de concurrencia, por lo que resulta fundamental evaluar aspectos como la sincronización, las condiciones de carrera, la exclusión mutua y la prevención de deadlocks. En conclusión, la inteligencia artificial generativa constituye una herramienta útil para asistir en el desarrollo de software concurrente, pero sus resultados requieren una revisión crítica y validación técnica.

VI. REFERENCIAS Y BIBLIOGRAFÍA

D. Hirschfeldt, "Apunte de Hilos (Thread)," Programación Concurrente, Universidad Nacional de La Matanza, s. f.

Cátedra de Programación Concurrente, "Problemas en concurrencia," Universidad Nacional de La Matanza, material de clase, s. f.

Cátedra de Programación Concurrente, "Comunicación entre procesos," Universidad Nacional de La Matanza, material de clase, s. f.